

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices,
transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

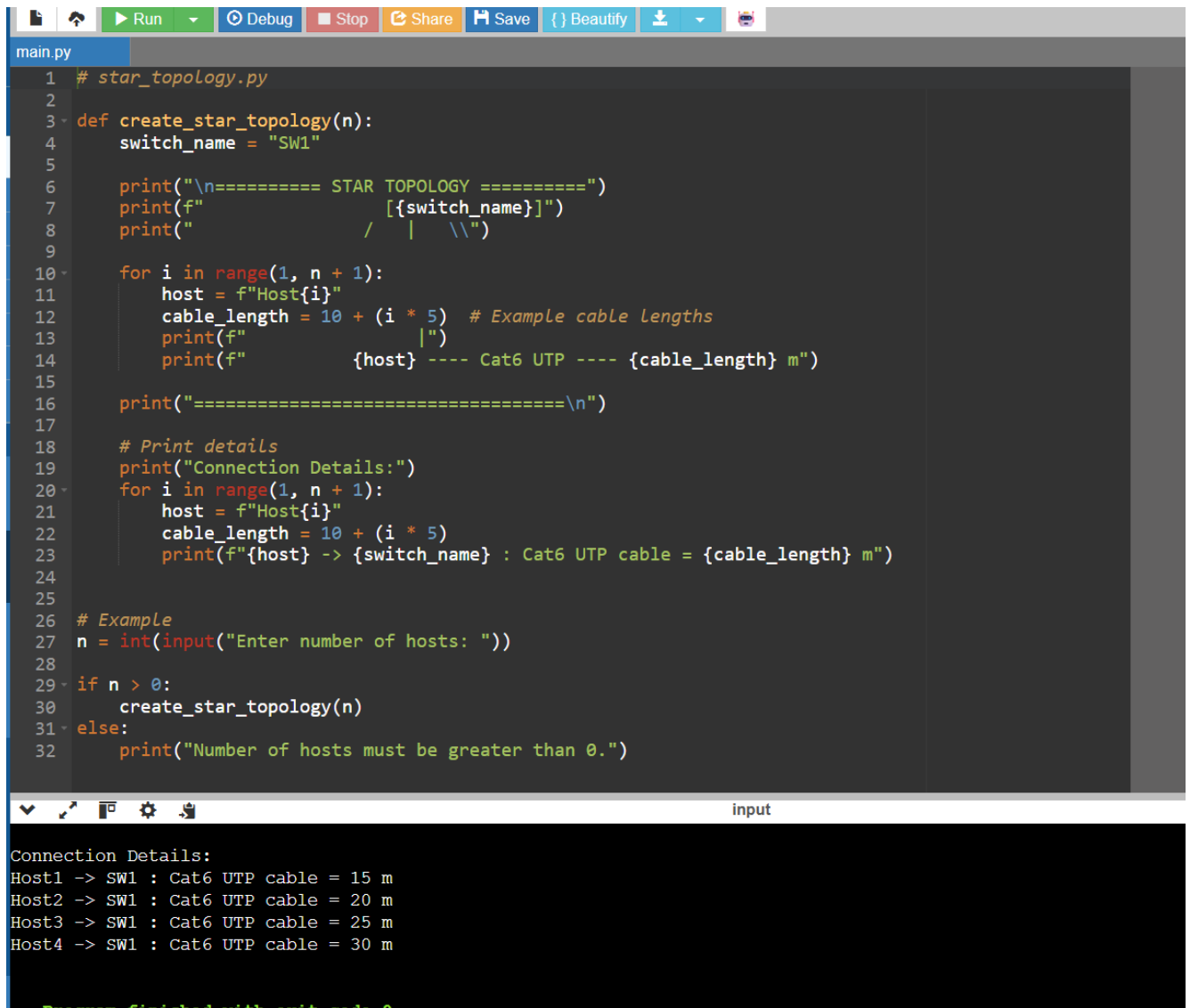
I **B Tharun Kumar / 192424356** (Name / Reg No) ~~certify that this submission is my~~
original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Tharun Kumar

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.



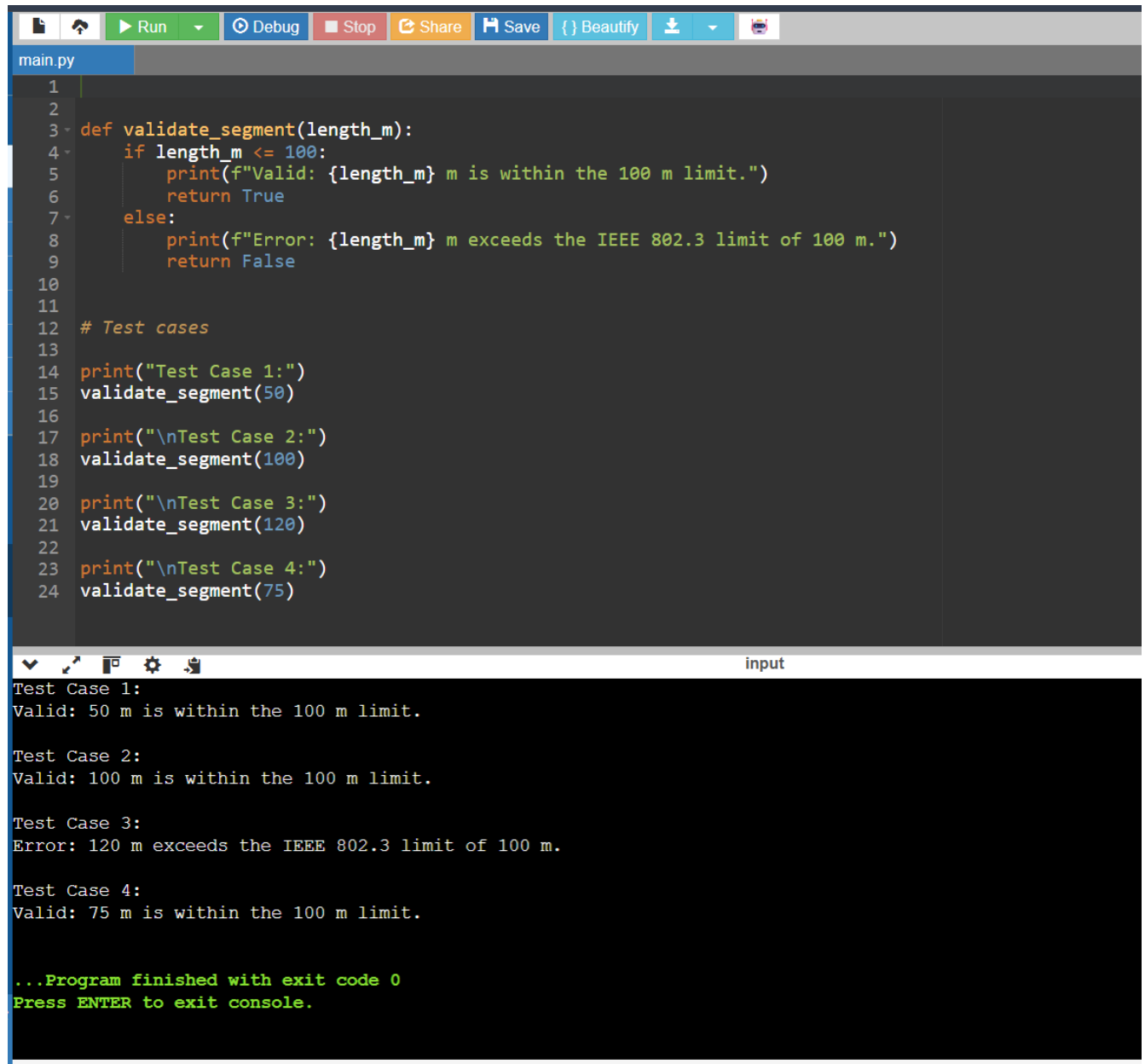
```
1 # star_topology.py
2
3 def create_star_topology(n):
4     switch_name = "SW1"
5
6     print("\n===== STAR TOPOLOGY =====")
7     print(f"          [{switch_name}]")
8     print("          /  |  \\\")
9
10    for i in range(1, n + 1):
11        host = f"Host{i}"
12        cable_length = 10 + (i * 5) # Example cable lengths
13        print(f"          |")
14        print(f"          {host} ---- Cat6 UTP ---- {cable_length} m")
15
16    print("=====\\n")
17
18    # Print details
19    print("Connection Details:")
20    for i in range(1, n + 1):
21        host = f"Host{i}"
22        cable_length = 10 + (i * 5)
23        print(f"{host} -> {switch_name} : Cat6 UTP cable = {cable_length} m")
24
25
26 # Example
27 n = int(input("Enter number of hosts: "))
28
29 if n > 0:
30     create_star_topology(n)
31 else:
32     print("Number of hosts must be greater than 0.")
```

input

```
Connection Details:
Host1 -> SW1 : Cat6 UTP cable = 15 m
Host2 -> SW1 : Cat6 UTP cable = 20 m
Host3 -> SW1 : Cat6 UTP cable = 25 m
Host4 -> SW1 : Cat6 UTP cable = 30 m

Program finished with exit code 0
```

2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.



The image shows a code editor window with a toolbar at the top containing icons for Run, Debug, Stop, Share, Save, Beautify, and a dropdown menu. The editor displays a Python file named `main.py` with the following code:

```
1
2
3 def validate_segment(length_m):
4     if length_m <= 100:
5         print(f"Valid: {length_m} m is within the 100 m limit.")
6         return True
7     else:
8         print(f"Error: {length_m} m exceeds the IEEE 802.3 limit of 100 m.")
9         return False
10
11
12 # Test cases
13
14 print("Test Case 1:")
15 validate_segment(50)
16
17 print("\nTest Case 2:")
18 validate_segment(100)
19
20 print("\nTest Case 3:")
21 validate_segment(120)
22
23 print("\nTest Case 4:")
24 validate_segment(75)
```

Below the editor is a console window with the following output:

```
Test Case 1:
Valid: 50 m is within the 100 m limit.

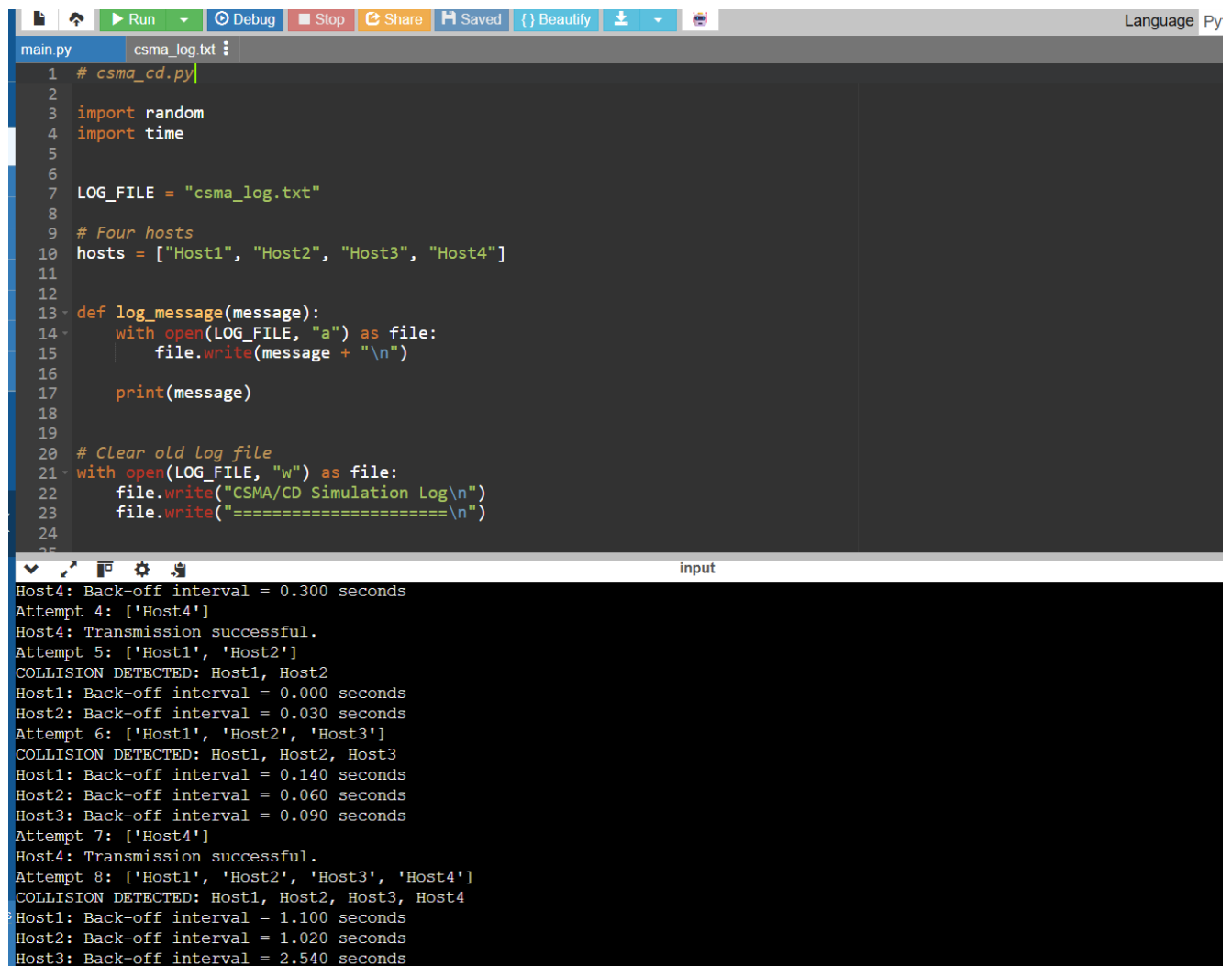
Test Case 2:
Valid: 100 m is within the 100 m limit.

Test Case 3:
Error: 120 m exceeds the IEEE 802.3 limit of 100 m.

Test Case 4:
Valid: 75 m is within the 100 m limit.

...Program finished with exit code 0
Press ENTER to exit console.
```

3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma\_log.txt`.

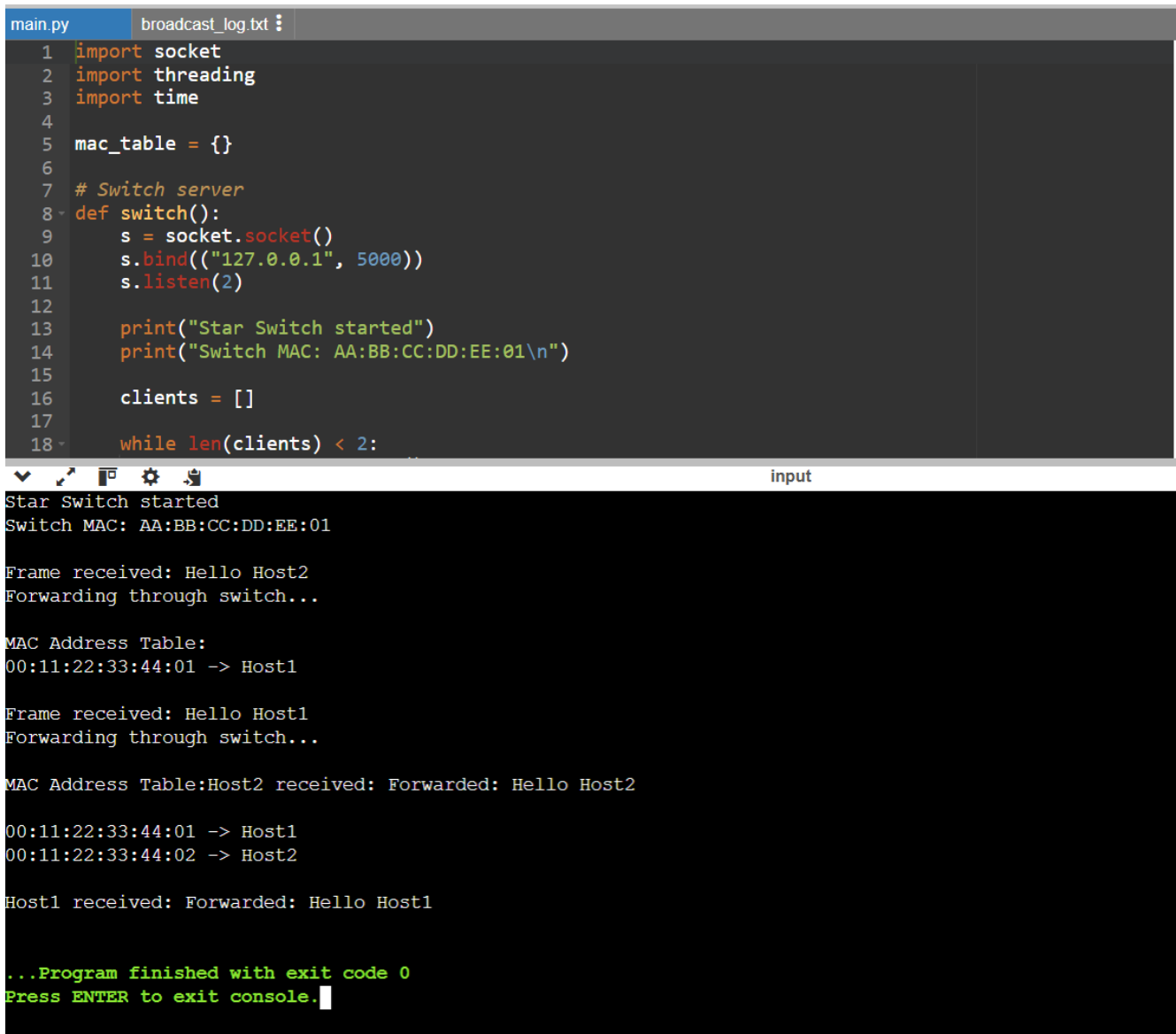


The image shows a Python IDE with a script for simulating CSMA/CD on a star topology with 4 hosts. The script is named `csma_cd.py` and is saved as `csma_log.txt`. The script includes imports for `random` and `time`, defines a log file path, and sets up four hosts: `Host1`, `Host2`, `Host3`, and `Host4`. It defines a `log_message` function to write to the log file and a main loop that simulates transmission attempts and collision detection. The output shows the simulation results, including back-off intervals and collision detection events.

```
1 # csma_cd.py
2
3 import random
4 import time
5
6
7 LOG_FILE = "csma_log.txt"
8
9 # Four hosts
10 hosts = ["Host1", "Host2", "Host3", "Host4"]
11
12
13 def log_message(message):
14     with open(LOG_FILE, "a") as file:
15         file.write(message + "\n")
16
17     print(message)
18
19
20 # Clear old log file
21 with open(LOG_FILE, "w") as file:
22     file.write("CSMA/CD Simulation Log\n")
23     file.write("=====\n")
24
25
```

Host4: Back-off interval = 0.300 seconds
Attempt 4: ['Host4']
Host4: Transmission successful.
Attempt 5: ['Host1', 'Host2']
COLLISION DETECTED: Host1, Host2
Host1: Back-off interval = 0.000 seconds
Host2: Back-off interval = 0.030 seconds
Attempt 6: ['Host1', 'Host2', 'Host3']
COLLISION DETECTED: Host1, Host2, Host3
Host1: Back-off interval = 0.140 seconds
Host2: Back-off interval = 0.060 seconds
Host3: Back-off interval = 0.090 seconds
Attempt 7: ['Host4']
Host4: Transmission successful.
Attempt 8: ['Host1', 'Host2', 'Host3', 'Host4']
COLLISION DETECTED: Host1, Host2, Host3, Host4
Host1: Back-off interval = 1.100 seconds
Host2: Back-off interval = 1.020 seconds
Host3: Back-off interval = 2.540 seconds

4. Using Python's 'socket' library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.



The image shows a code editor with a file named `main.py` and a terminal window below it. The code in `main.py` is a Python script that simulates a star switch. It imports `socket`, `threading`, and `time`. It defines a `switch()` function that creates a socket, binds it to `127.0.0.1` on port `5000`, and listens for two connections. The switch maintains a `mac_table` dictionary and a `clients` list. The terminal output shows the switch starting, receiving frames from Host2 and Host1, forwarding them, and updating the MAC address table. The program ends with an exit code of 0.

```
main.py broadcast_log.txt
1 import socket
2 import threading
3 import time
4
5 mac_table = {}
6
7 # Switch server
8 def switch():
9     s = socket.socket()
10    s.bind(("127.0.0.1", 5000))
11    s.listen(2)
12
13    print("Star Switch started")
14    print("Switch MAC: AA:BB:CC:DD:EE:01\n")
15
16    clients = []
17
18    while len(clients) < 2:
```

```
Star Switch started
Switch MAC: AA:BB:CC:DD:EE:01

Frame received: Hello Host2
Forwarding through switch...

MAC Address Table:
00:11:22:33:44:01 -> Host1

Frame received: Hello Host1
Forwarding through switch...

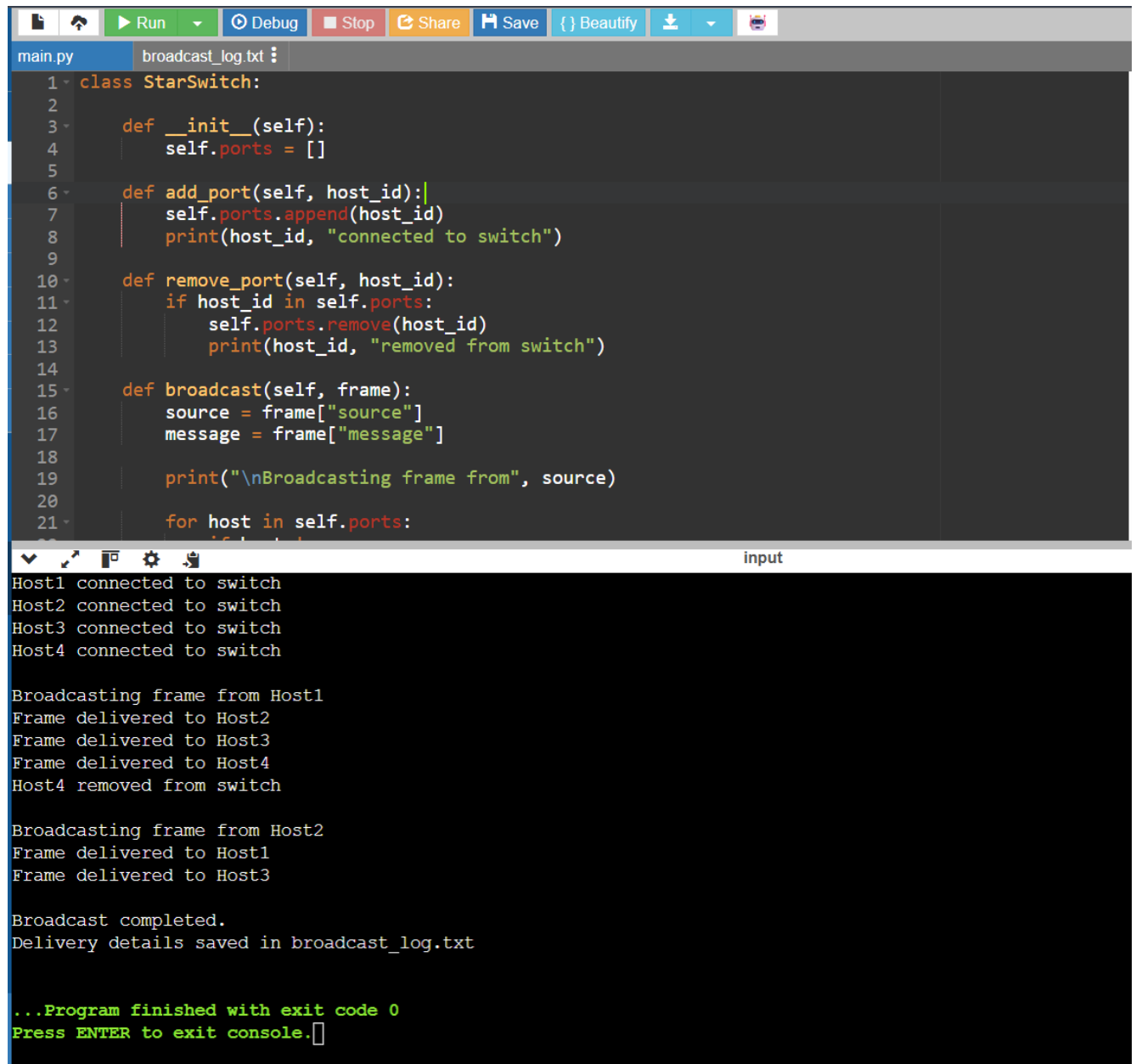
MAC Address Table: Host2 received: Forwarded: Hello Host2

00:11:22:33:44:01 -> Host1
00:11:22:33:44:02 -> Host2

Host1 received: Forwarded: Hello Host1

...Program finished with exit code 0
Press ENTER to exit console.
```

5. Create a class `StarSwitch` with methods `add\_port(host\_id)`, `remove\_port(host\_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.



```
main.py broadcast_log.txt
1 class StarSwitch:
2
3     def __init__(self):
4         self.ports = []
5
6     def add_port(self, host_id):
7         self.ports.append(host_id)
8         print(host_id, "connected to switch")
9
10    def remove_port(self, host_id):
11        if host_id in self.ports:
12            self.ports.remove(host_id)
13            print(host_id, "removed from switch")
14
15    def broadcast(self, frame):
16        source = frame["source"]
17        message = frame["message"]
18
19        print("\nBroadcasting frame from", source)
20
21        for host in self.ports:
```

input

```
Host1 connected to switch
Host2 connected to switch
Host3 connected to switch
Host4 connected to switch

Broadcasting frame from Host1
Frame delivered to Host2
Frame delivered to Host3
Frame delivered to Host4
Host4 removed from switch

Broadcasting frame from Host2
Frame delivered to Host1
Frame delivered to Host3

Broadcast completed.
Delivery details saved in broadcast_log.txt

...Program finished with exit code 0
Press ENTER to exit console.
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done